

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Coding Challenge Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 - Coding Challenge Task
Weightage:	40% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	S.SRI MATHI	Reg. No.:	192424095
Section / Batch:	SLOT B	Date:	19.08.2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Write clear code for the given problem.
- Analyze the Time complexity and Space complexity of your code using dynamic programming and greedy strategies

Question Type	Focus Area	Questions
Coding Challenge Task	Focus on Code and analyze optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Coding Challenge Task

Code of Conduct

I S.SRI MATHI (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Sri Mathi

Signature of the Student:

RUBRICS FOR CALCULATION

Coding Challenge Task

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%				
Algorithm	25%				
Code Implementation	20%				
Competitive Performance	20%				
Test Case Handling	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name :	Name :	Name :
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

CO4-AT1

Coding Challenge Task

NAME: S.SRI MATHI

REG.NO: 192424095

COURSE CODE: CSA0603

COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHM

DATE: 19.08.2026

Q35. Minimum Path Sum in Triangle

A triangle of numbers is given, and the goal is to find the minimum path sum from top to bottom. At each step, you may move to adjacent numbers in the next row. The objective is to determine the minimum total sum. Use Dynamic Programming to solve efficiently.

Sample Input:

triangle =

2

3 4

6 5 7

Expected Output:

Min Sum = 10

Minimum Path Sum in Triangle

Aim

To implement the **Minimum Path Sum in a Triangle** problem using **Dynamic Programming** and visualize the step-by-step computation to find the minimum sum path from the top to the bottom of the triangle.

Objective

- To understand the concept of **Dynamic Programming**.
- To find the minimum possible path sum from the top to the bottom of a number triangle.
- To use a **bottom-up DP approach** to solve the problem efficiently.
- To store previously calculated results and avoid repeated calculations.
- To identify and display the actual minimum path.
- To provide an interactive visualization using **Back, Play/Pause, and Next** controls.

Minimum Path Sum in Triangle

Minimum Path Sum in Triangle is an **optimization problem** where the goal is to find the path from the top to the bottom of a number triangle with the **minimum total sum**.

At each step, we can move to either of the **two adjacent numbers in the next row**.

Example:

```
    2
   /\
  3 4
 /\ \
6 5 7
```

The minimum path is:

$2 \rightarrow 3 \rightarrow 5 = 10$

Dynamic Programming is used by solving the problem **bottom-up**. For each element, we add its value to the smaller of its two children:

$$DP[i][j] = \text{triangle}[i][j] + \min(DP[i+1][j], DP[i+1][j+1])$$

This avoids repeated calculations and efficiently finds the minimum path sum.

Algorithm

Input: A triangle of numbers.

Output: Minimum path sum from top to bottom.

Step 1: Read the triangle.

Step 2: Copy the triangle into a DP table.

Step 3: Start from the second-last row.

Step 4: For every element in the current row:

- a. Get its two children.
- b. Find the minimum of the two children.
- c. Add this minimum to the current element.
- d. Store the result in DP.

Step 5: Move to the previous row.

Step 6: Repeat until the top row is reached.

Step 7: The top DP value is the minimum path sum.

Step 8: Trace the minimum child at each level
to find the actual minimum path.

Step 9: Display the minimum path and minimum sum.

Pseudocode

MINIMUM_PATH_SUM(triangle)

dp $\leftarrow$ copy of triangle

for i $\leftarrow$ last row - 1 down to 0

for j $\leftarrow$ 0 to length(triangle[i]) - 1

dp[i][j] $\leftarrow$ triangle[i][j]
+ min(dp[i+1][j],
dp[i+1][j+1])

minimum_sum $\leftarrow$ dp[0][0]

return minimum_sum

Path Reconstruction

i $\leftarrow$ 0

j $\leftarrow$ 0

path $\leftarrow$ triangle[0][0]

while i < last row

if dp[i+1][j] <= dp[i+1][j+1]

choose left child

else

choose right child

move to next row

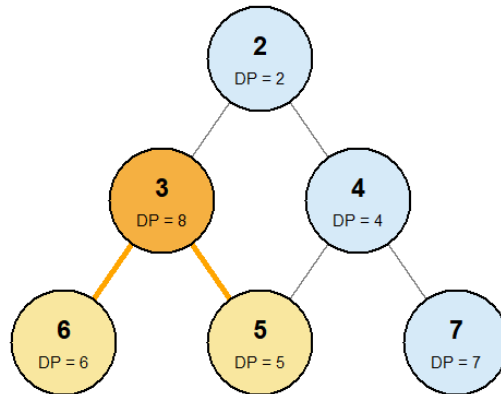
add selected value to path

return path

Output

Minimum Path Sum - Dynamic Programming

Minimum Path Sum in Triangle



STEP 1 / 3

Current Computation

Compare children:
 $\min(6, 5) = 5$

Current value: 3
 $3 + 5 = 8$

FINAL ANSWER

$2 \rightarrow 3 \rightarrow 5$

Min Sum = 10

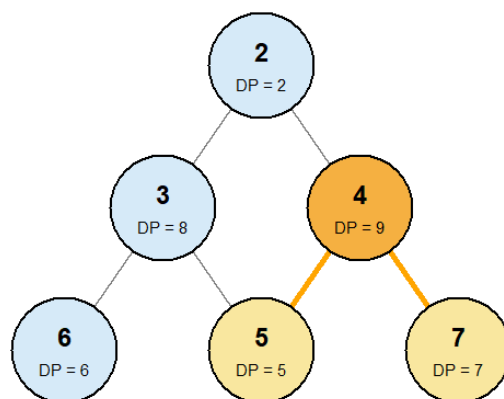
Back

Play

Next

Minimum Path Sum - Dynamic Programming

Minimum Path Sum in Triangle



STEP 2 / 3

Current Computation

Compare children:
 $\min(5, 7) = 5$

Current value: 4
 $4 + 5 = 9$

FINAL ANSWER

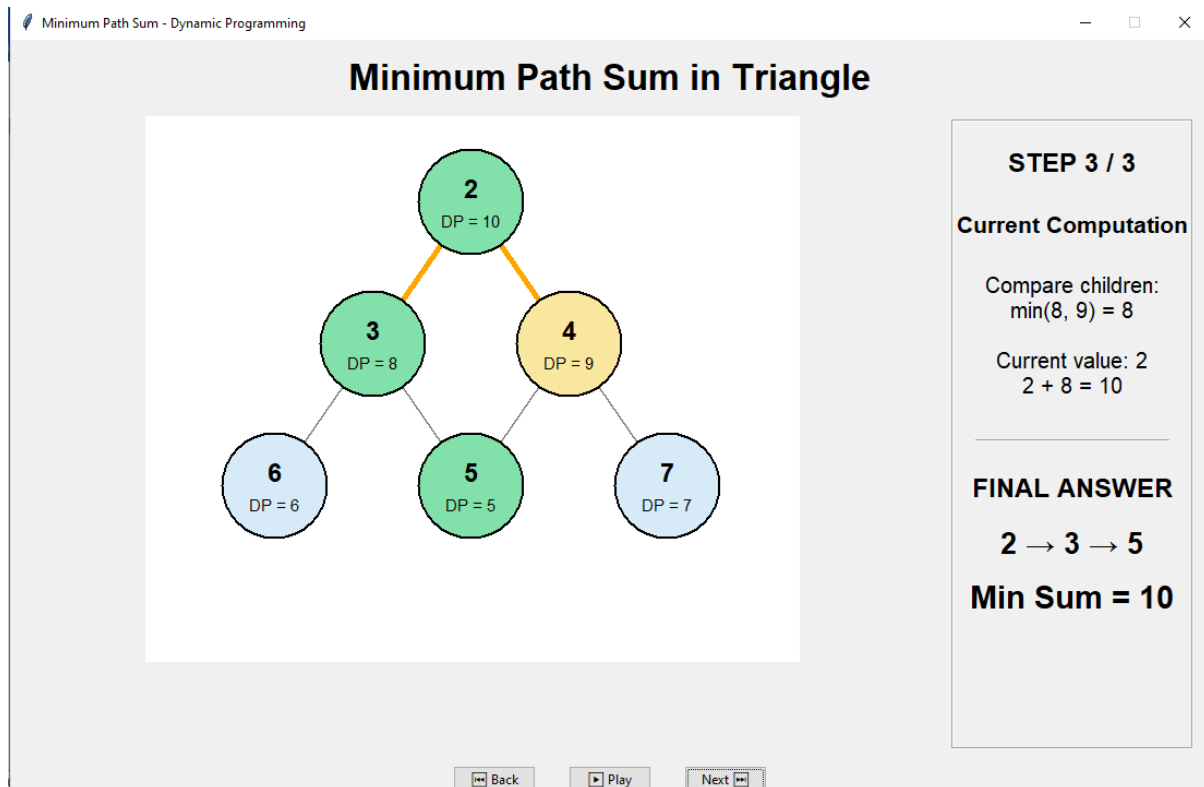
$2 \rightarrow 3 \rightarrow 5$

Min Sum = 10

Back

Play

Next



Conclusion

The **Minimum Path Sum in Triangle** was successfully solved using the **Dynamic Programming** technique. The bottom-up approach calculates the minimum cost for each position by choosing the smaller of its two adjacent children. This avoids unnecessary repeated calculations and efficiently obtains the minimum path sum.

For the given input, the minimum path is:

$2 \rightarrow 3 \rightarrow 5$

and the minimum path sum is:

10

The interactive visualization makes the DP process easier to understand by displaying each computation step and allowing the user to move **backward, forward, or play/pause** the process.

